

U3 S14: Tarea 1

Integrantes:

Anchundia Caicedo Naldo Jonnel

Aviles Baque Pedro Andrés

Félix Javier Tomalá González

Franco Quimi Víctor German

López Moreno Adiel Stalin

Facultad de Ciencias Matemáticas y Físicas

Universidad de Guayaquil

Carrera de Software

Gestión de la Configuración del Software

SOF-S-NO-9-5

Ing. Parrales Bravo Franklin Ricardo

Guayas, Ecuador

21 de noviembre de 2025

Contenido

Informe	3
1. Introducción.....	3
2. Objetivos del Proyecto.....	3
2.1 Objetivo General.....	3
2.2 Objetivos Específicos	3
3. Descripción General del Proyecto.....	3
4. Configuración del Proyecto Maven	4
5. Gestión de Dependencias.....	4
5.1 JUnit – Framework para Pruebas Unitarias.....	5
5.2 Log4j – Biblioteca para Registro de Eventos	5
5.3 Gson – Manejo de Datos en Formato JSON	6
6. Ciclo de Vida del Software con Maven	6
6.1 Fase validate	6
6.2 Fase test	7
6.3 Fase package.....	7
6.4 Fase install	8
7. Implementación de Pruebas Unitarias.....	8
8. Resultados y Análisis.....	9
9. Manual de Operaciones	10
Requisitos del Sistema	10
Procedimiento de Ejecución	11
10. Reflexión Final	11
11. Anexos.....	11

Informe

1. Introducción

La gestión de la configuración del software es una disciplina esencial dentro de la ingeniería de software, ya que permite controlar, organizar y mantener de manera ordenada los diferentes elementos que componen un sistema informático a lo largo de su ciclo de vida. Esta práctica resulta fundamental para garantizar la estabilidad, mantenibilidad y trazabilidad del software, especialmente en proyectos que evolucionan constantemente.

En el desarrollo de aplicaciones modernas, la correcta administración de dependencias y la automatización de procesos como la compilación, ejecución de pruebas y empaquetado se han convertido en una necesidad. Herramientas como **Apache Maven** permiten estandarizar estos procesos, reduciendo errores manuales y mejorando la eficiencia del equipo de desarrollo.

El presente proyecto tiene como finalidad aplicar los conceptos de gestión de la configuración del software mediante el uso de Maven, configurando un proyecto Java estructurado que permita gestionar dependencias externas, ejecutar pruebas unitarias y automatizar el ciclo de vida del software, utilizando el entorno de desarrollo IntelliJ IDEA.

2. Objetivos del Proyecto

2.1 Objetivo General

Aplicar los principios de la gestión de la configuración del software mediante el uso de Apache Maven, desarrollando un proyecto Java que integre la gestión de dependencias, la ejecución de pruebas unitarias y la automatización del ciclo de vida del software.

2.2 Objetivos Específicos

- Comprender la estructura estándar de un proyecto Maven y su importancia en el desarrollo de software.
- Configurar correctamente el archivo `pom.xml` para la gestión de dependencias.
- Ejecutar las principales fases del ciclo de vida de Maven.
- Implementar pruebas unitarias básicas como mecanismo de aseguramiento de la calidad.
- Documentar el proceso de configuración, ejecución y resultados obtenidos.

3. Descripción General del Proyecto

El proyecto desarrollado corresponde a una aplicación Java básica configurada como un proyecto Maven. Su propósito principal no es la complejidad funcional, sino la correcta aplicación de los conceptos de gestión de configuración del software.

La estructura del proyecto sigue el estándar establecido por Maven, separando claramente el código fuente principal de las pruebas unitarias. Esta organización facilita el mantenimiento del proyecto y permite que las herramientas de automatización identifiquen correctamente cada componente durante el proceso de construcción.

4. Configuración del Proyecto Maven

La configuración del proyecto se realizó utilizando Apache Maven, herramienta que permite definir de forma centralizada la información del proyecto y sus dependencias. El archivo principal de configuración es el `pom.xml`, en el cual se establecen los identificadores únicos del proyecto como el `groupId`, `artifactId` y la versión.

Estos identificadores permiten que Maven reconozca el proyecto dentro de su repositorio local y facilitan la gestión de versiones del software. Además, se definieron propiedades relacionadas con la versión del lenguaje Java utilizado, garantizando compatibilidad durante el proceso de compilación.

A screenshot of a code editor window with a dark background and a blue border. The window has three colored window control buttons (red, yellow, green) in the top-left corner and a plus sign in the top-right corner. The code displayed is XML configuration for a Maven project, with syntax highlighting:

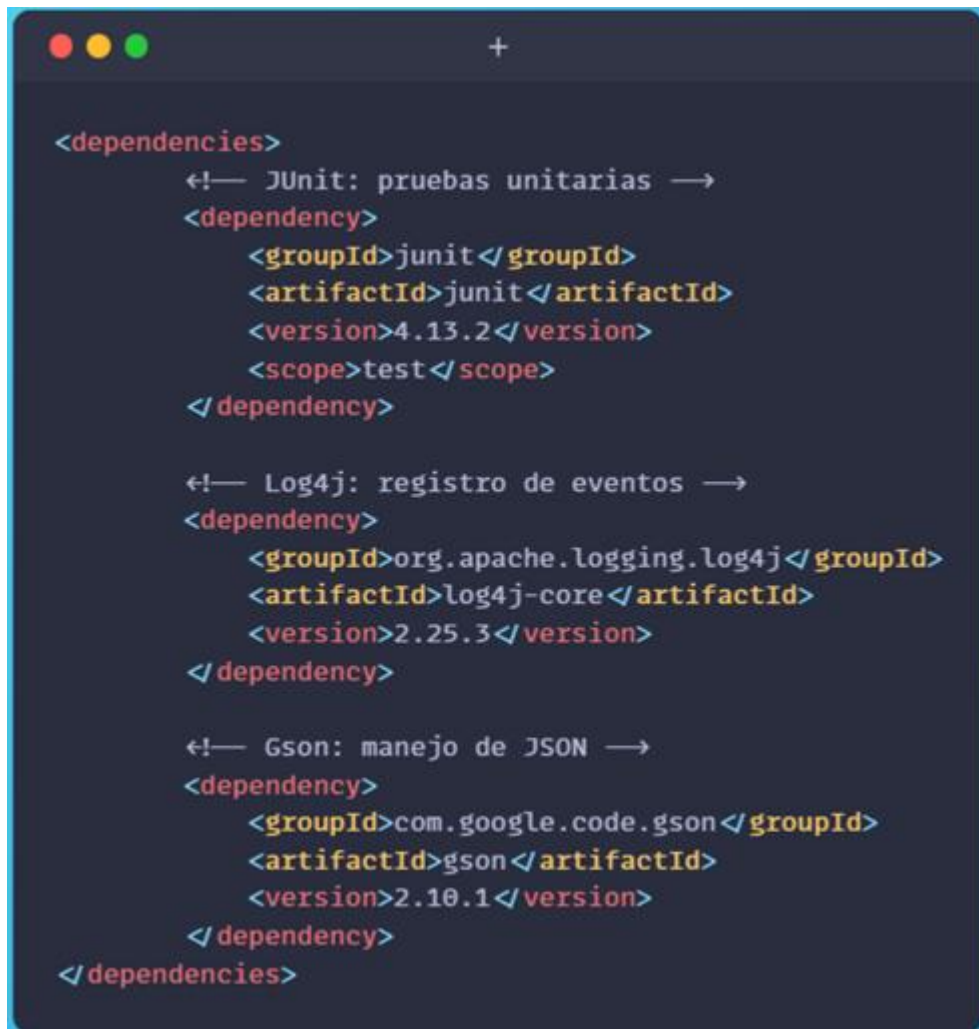
```
<groupId>ec.edu.tuuniversidad</groupId>
<artifactId>gestion-configuracion-software</artifactId>
<version>1.0-SNAPSHOT</version>
```

Captura del archivo `pom.xml` mostrando `groupId`, `artifactId`, `version` y propiedades del compilador.

5. Gestión de Dependencias

Uno de los principales beneficios de Maven es la gestión automática de dependencias. En lugar de incluir manualmente librerías externas en el proyecto, Maven permite declararlas en el archivo `pom.xml`, descargándolas automáticamente desde repositorios remotos.

En este proyecto se incorporaron dependencias externas que cumplen diferentes funciones dentro del sistema, tales como la ejecución de pruebas unitarias, el registro de eventos del sistema y el manejo de datos estructurados. Esta gestión centralizada mejora la organización del proyecto y reduce conflictos entre versiones de librerías.

A screenshot of a code editor window with a dark theme. The window has three colored window control buttons (red, yellow, green) in the top-left corner and a plus sign in the top-right corner. The code is XML, specifically the <dependencies> section of a pom.xml file. It lists three dependencies: JUnit 4.13.2, Log4j 2.25.3, and Gson 2.10.1. Each dependency is enclosed in a <dependency> tag, and the entire section is enclosed in <dependencies> and </dependencies> tags. Comments in Spanish are placed above each dependency block.

```
<dependencies>
  <!-- JUnit: pruebas unitarias -->
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.13.2</version>
    <scope>test</scope>
  </dependency>

  <!-- Log4j: registro de eventos -->
  <dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-core</artifactId>
    <version>2.25.3</version>
  </dependency>

  <!-- Gson: manejo de JSON -->
  <dependency>
    <groupId>com.google.code.gson</groupId>
    <artifactId>gson</artifactId>
    <version>2.10.1</version>
  </dependency>
</dependencies>
```

Captura de la sección <dependencies> del archivo pom.xml.

5.1 JUnit – Framework para Pruebas Unitarias

JUnit es un framework ampliamente utilizado en el desarrollo de aplicaciones Java para la creación y ejecución de pruebas unitarias. Su principal objetivo es permitir la validación del comportamiento de métodos y componentes de manera aislada, asegurando que cada parte del sistema funcione correctamente.

En este proyecto, JUnit fue utilizado para implementar pruebas unitarias básicas que verifican operaciones lógicas y matemáticas simples. Estas pruebas permiten detectar errores en etapas tempranas del desarrollo y garantizan que los cambios realizados en el código no afecten negativamente su funcionamiento.

La inclusión de JUnit como dependencia facilita la ejecución automatizada de pruebas mediante el comando `mvn test`, integrando el aseguramiento de la calidad dentro del ciclo de vida del software.

5.2 Log4j – Biblioteca para Registro de Eventos

Log4j es una biblioteca utilizada para el registro de eventos y mensajes durante la ejecución de una aplicación. El uso de un sistema de logging permite monitorear el

comportamiento del software, registrar información relevante y facilitar la identificación de errores o situaciones anómalas.

En el proyecto, Log4j fue incorporado como dependencia para gestionar mensajes informativos durante la ejecución del programa. Esta biblioteca permite registrar eventos de manera estructurada y configurable, mejorando la trazabilidad del sistema y facilitando tareas de depuración y mantenimiento.

La integración de Log4j dentro del proyecto demuestra el uso de buenas prácticas en el desarrollo de software, ya que el registro de eventos es una herramienta clave para el análisis del comportamiento de las aplicaciones en entornos reales.

5.3 Gson – Manejo de Datos en Formato JSON

Gson es una librería desarrollada por Google que permite convertir objetos Java a formato JSON y viceversa. Este tipo de funcionalidad es ampliamente utilizada en aplicaciones modernas que intercambian información de forma estructurada, especialmente en servicios web y aplicaciones distribuidas.

En este proyecto, la dependencia Gson fue incluida para demostrar el uso de bibliotecas externas destinadas al manejo de datos. Su incorporación permite convertir información interna del programa en formato JSON, facilitando la interoperabilidad y el procesamiento de datos.

El uso de Gson complementa la gestión de dependencias del proyecto, evidenciando cómo Maven permite integrar fácilmente librerías externas para ampliar las capacidades del software sin afectar su estructura interna.

6. Ciclo de Vida del Software con Maven

Maven define un ciclo de vida compuesto por distintas fases que representan las etapas por las que pasa un proyecto durante su construcción. Estas fases permiten automatizar tareas clave del desarrollo de software.

En el presente proyecto se ejecutaron las fases principales del ciclo de vida de Maven, las cuales permitieron validar la estructura del proyecto, ejecutar pruebas unitarias, generar el artefacto final e instalarlo en el repositorio local.

La ejecución correcta de estas fases confirma que el proyecto cumple con los estándares necesarios para su construcción y distribución.

6.1 Fase validate

La fase **validate** tiene como objetivo verificar que el proyecto esté correctamente estructurado y que cumpla con los requisitos mínimos necesarios para su construcción. Durante esta fase, Maven comprueba que el archivo `pom.xml` esté correctamente definido y que la estructura del proyecto siga el estándar esperado.

Esta fase no compila el código ni ejecuta pruebas, sino que actúa como una validación inicial que permite detectar errores de configuración antes de avanzar a etapas posteriores del ciclo de vida.

```
PS C:\Users\USER\Documents\Proyectos_Pequeños\Proyecto> mvn validate
WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe::staticFieldBase has been called by com.google.inject.internal.aop.HiddenClassDefiner (file:/C:/Program%20Files/Apache/maven/lib/guice-5.1.0-classes.jar)
WARNING: Please consider reporting this to the maintainers of class com.google.inject.internal.aop.HiddenClassDefiner
WARNING: sun.misc.Unsafe::staticFieldBase will be removed in a future release
[INFO] Scanning for projects...
[INFO] -----< ec.edu.tuuniversidad:gestion-configuracion-software >-----
[INFO] Building gestion-configuracion-software 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 0.341 s
[INFO] Finished at: 2026-02-01T12:09:54-05:00
[INFO] -----
```

Captura de la ejecución del comando `mvn validate` con resultado exitoso.

6.2 Fase test

La fase **test** se encarga de ejecutar las pruebas unitarias definidas en el proyecto. Durante esta etapa, Maven compila el código necesario para las pruebas y ejecuta automáticamente los métodos marcados como pruebas utilizando el framework correspondiente.

La ejecución de esta fase permite verificar que las funcionalidades del sistema se comporten de manera correcta y que los cambios realizados en el código no introduzcan errores. En este proyecto, esta fase fue utilizada para ejecutar pruebas unitarias básicas, obteniendo resultados satisfactorios.

```
PS C:\Users\USER\Documents\Proyectos_Pequeños\Proyecto> mvn test
WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe::staticFieldBase has been called by com.google.inject.internal.aop.HiddenClassDefiner (file:/C:/Program%20Files/Apache/maven/lib/guice-5.1.0-classes.jar)
WARNING: Please consider reporting this to the maintainers of class com.google.inject.internal.aop.HiddenClassDefiner
WARNING: sun.misc.Unsafe::staticFieldBase will be removed in a future release
[INFO] Scanning for projects...
[INFO] -----< ec.edu.tuuniversidad:gestion-configuracion-software >-----
[INFO] Building gestion-configuracion-software 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO] --- resources:3.3.1:resources (default-resources) @ gestion-configuracion-software ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory C:\Users\USER\Documents\Proyectos_Pequeños\Proyecto\src/main/resources
[INFO] --- compiler:3.13.0:compile (default-compile) @ gestion-configuracion-software ---
[INFO] Nothing to compile - all classes are up to date.
[INFO] --- resources:3.3.1:testResources (default-testResources) @ gestion-configuracion-software ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory C:\Users\USER\Documents\Proyectos_Pequeños\Proyecto\src/test/resources
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ gestion-configuracion-software ---
[INFO] Nothing to compile - all classes are up to date.
[INFO] --- surefire:3.2.5:test (default-test) @ gestion-configuracion-software ---
[INFO] Using auto detected provider org.apache.maven.surefire.junit4.JUnit4Provider
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running ec.edu.tuuniversidad.AppTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.213 s -- in ec.edu.tuuniversidad.AppTest
[INFO] Results:
[INFO]
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 0.680 s
[INFO] Finished at: 2026-02-01T12:10:12-05:00
[INFO] -----
```

Captura de la ejecución del comando `mvn test` mostrando las pruebas unitarias ejecutadas correctamente.

6.3 Fase package

La fase **package** se encarga de compilar el código fuente del proyecto y generar un artefacto distribuible, generalmente en formato JAR. Durante esta fase, Maven ejecuta todas las

fases anteriores del ciclo de vida de forma automática, asegurando que el proyecto haya sido validado y probado antes de ser empaquetado.

El resultado de esta fase es la creación del archivo JAR dentro de la carpeta `target`, el cual representa la versión compilada del proyecto lista para ser distribuida o ejecutada.

```
PS C:\Users\USER\Documents\Proyectos_Pequeños\Proyecto> mvn package
WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe::staticFieldBase has been called by com.google.inject.internal.aop.HiddenClassDefiner (file:/C:/Program%20Files/Apache/maven/lib/guice-5.1.0-classes.jar)
WARNING: Please consider reporting this to the maintainers of class com.google.inject.internal.aop.HiddenClassDefiner
WARNING: sun.misc.Unsafe::staticFieldBase will be removed in a future release
[INFO] Scanning for projects...
[INFO]
[INFO] -----< ec.edu.tuuniversidad:gestion-configuracion-software >-----
[INFO] Building gestion-configuracion-software 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ gestion-configuracion-software ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory C:\Users\USER\Documents\Proyectos_Pequeños\Proyecto\src\main\resources
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ gestion-configuracion-software ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ gestion-configuracion-software ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory C:\Users\USER\Documents\Proyectos_Pequeños\Proyecto\src\test\resources
[INFO]
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ gestion-configuracion-software ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- surefire:3.2.5:test (default-test) @ gestion-configuracion-software ---
[INFO] Using auto detected provider org.apache.maven.surefire.junit4.JUnit4Provider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running ec.edu.tuuniversidad.AppTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.252 s -- in ec.edu.tuuniversidad.AppTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jar:3.4.1:jar (default-jar) @ gestion-configuracion-software ---
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] -----
[INFO] Total time: 7.383 s
[INFO] Finished at: 2026-02-01T12:10:40-05:00
[INFO]
1:11 PM
```

Captura de la carpeta `target` mostrando el archivo JAR generado.

6.4 Fase install

La fase **install** se encarga de instalar el artefacto generado en el repositorio local de Maven. Este repositorio permite que otros proyectos locales puedan utilizar el artefacto como dependencia sin necesidad de descargarlo nuevamente desde un repositorio remoto.

La ejecución de esta fase confirma que el proyecto ha sido construido correctamente y que se encuentra disponible para su reutilización dentro del entorno local de desarrollo.

```
PS C:\Users\USER\Documents\Proyectos_Pequeños\Proyecto> mvn install
WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe::staticFieldBase has been called by com.google.inject.internal.aop.HiddenClassDefiner (file:/C:/Program%20Files/Apache/maven/lib/guice-5.1.0-classes.jar)
WARNING: Please consider reporting this to the maintainers of class com.google.inject.internal.aop.HiddenClassDefiner
WARNING: sun.misc.Unsafe::staticFieldBase will be removed in a future release
[INFO] Scanning for projects...
[INFO]
[INFO] -----< ec.edu.tuuniversidad:gestion-configuracion-software >-----
[INFO] Building gestion-configuracion-software 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ gestion-configuracion-software ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory C:\Users\USER\Documents\Proyectos_Pequeños\Proyecto\src\main\resources
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ gestion-configuracion-software ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ gestion-configuracion-software ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory C:\Users\USER\Documents\Proyectos_Pequeños\Proyecto\src\test\resources
[INFO]
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ gestion-configuracion-software ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- surefire:3.2.5:test (default-test) @ gestion-configuracion-software ---
[INFO] Using auto detected provider org.apache.maven.surefire.junit4.JUnit4Provider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running ec.edu.tuuniversidad.AppTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.216 s -- in ec.edu.tuuniversidad.AppTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jar:3.4.1:jar (default-jar) @ gestion-configuracion-software ---
[INFO]
[INFO] --- install:3.1.2:install (default-install) @ gestion-configuracion-software ---
[INFO] Installing C:\Users\USER\Documents\Proyectos_Pequeños\Proyecto\target\gestion-configuracion-software-1.0-SNAPSHOT\gestion-configuracion-software-1.0-SNAPSHOT.jar to C:\Users\USER\.m2\repository\ec/edu/tuuniversidad/gestion-configuracion-software/1.0-SNAPSHOT/gestion-configuracion-software-1.0-SNAPSHOT.jar
[INFO] Installing C:\Users\USER\Documents\Proyectos_Pequeños\Proyecto\target\gestion-configuracion-software-1.0-SNAPSHOT\gestion-configuracion-software-1.0-SNAPSHOT.pom to C:\Users\USER\.m2\repository\ec/edu/tuuniversidad/gestion-configuracion-software/1.0-SNAPSHOT/gestion-configuracion-software-1.0-SNAPSHOT.pom
[INFO] BUILD SUCCESS
[INFO]
[INFO] -----
[INFO] Total time: 7.392 s
[INFO] Finished at: 2026-02-01T12:10:55-05:00
[INFO]
1:12 PM 01/02/26
```

Captura de la ejecución del comando `mvn install` con resultado exitoso.

7. Implementación de Pruebas Unitarias

Las pruebas unitarias constituyen una práctica fundamental para garantizar la calidad del software. En este proyecto se implementaron pruebas unitarias básicas con el objetivo de verificar el correcto funcionamiento de métodos y operaciones simples del sistema.

La ejecución de estas pruebas permitió identificar posibles errores en etapas tempranas del desarrollo, asegurando que los cambios realizados no afecten negativamente el comportamiento del sistema. Las pruebas fueron ejecutadas exitosamente mediante las herramientas proporcionadas por Maven.



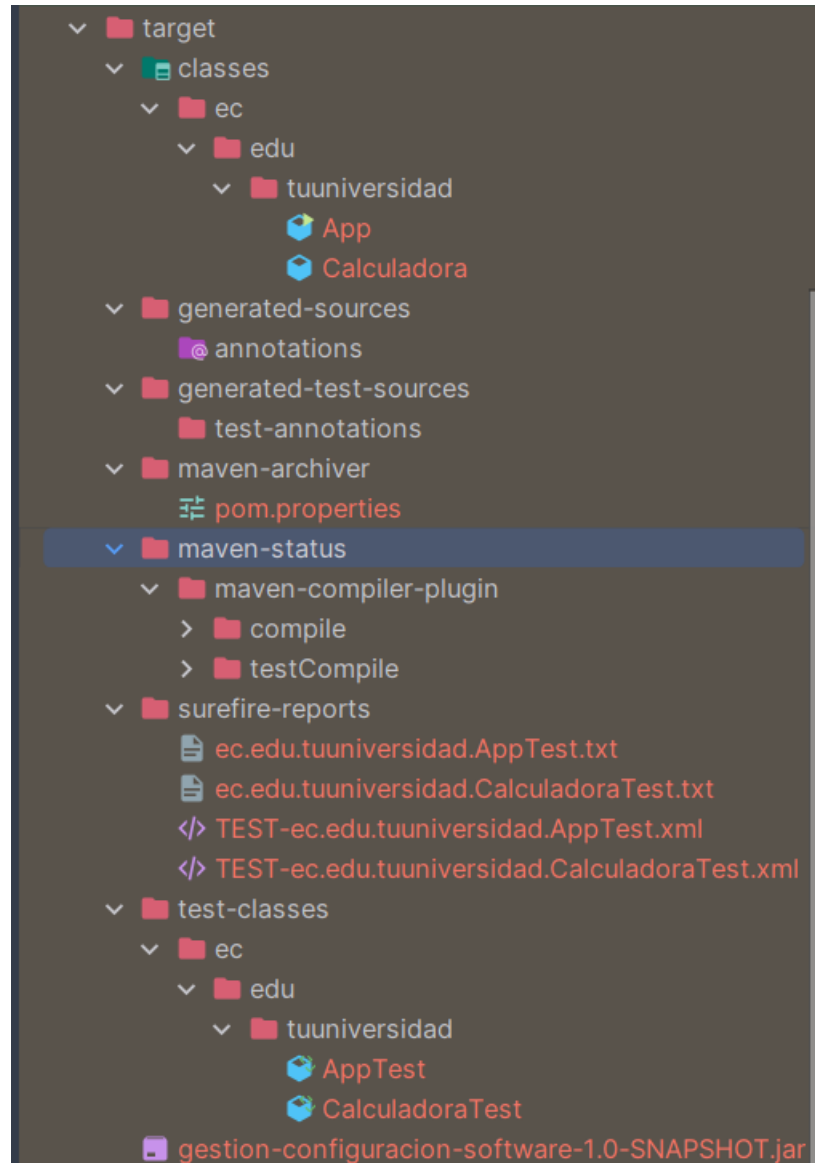
```
1 package ec.edu.tuuniversidad;
2
3 import static org.junit.Assert.*;
4 import org.junit.Before;
5 import org.junit.Test;
6
7 public class CalculadoraTest {
8     private Calculadora calculadora;
9
10    @Before
11    public void setUp() {
12        calculadora = new Calculadora();
13    }
14
15    @Test
16    public void testSuma() {
17        int resultado = calculadora.sumar(4, 6);
18        assertEquals(10, resultado);
19    }
20
21    @Test
22    public void testResta() {
23        int resultado = calculadora.restar(10, 3);
24        assertEquals(7, resultado);
25    }
26
27    @Test
28    public void testNumeroPar() {
29        assertTrue(calculadora.esPar(8));
30    }
31
32    @Test
33    public void testNumeroImpar() {
34        assertFalse(calculadora.esPar(7));
35    }
36
37 }
```

Captura de la ejecución de las pruebas unitarias en IntelliJ IDEA o salida del comando `mvn test`.

8. Resultados y Análisis

Como resultado del desarrollo del proyecto, se obtuvo una aplicación correctamente configurada como proyecto Maven. Las dependencias fueron gestionadas automáticamente y las pruebas unitarias se ejecutaron sin errores.

Asimismo, se generó el artefacto del proyecto dentro de la carpeta `target`, demostrando el correcto funcionamiento del proceso de compilación y empaquetado. Estos resultados evidencian la eficacia de Maven como herramienta de gestión de la configuración del software.



Captura de la carpeta `target` y del archivo JAR generado.

9. Manual de Operaciones

Requisitos del Sistema

- Java JDK 17 o superior
- Apache Maven
- Entorno de desarrollo compatible (IntelliJ IDEA)

Procedimiento de Ejecución

1. Clonar el repositorio del proyecto desde GitHub.
2. Abrir el proyecto en el entorno de desarrollo.
3. Ejecutar los comandos del ciclo de vida de Maven:

```
mvn validate
mvn test
mvn package
mvn install
```

4. Verificar la generación del artefacto en la carpeta `target`.

10. Reflexión Final

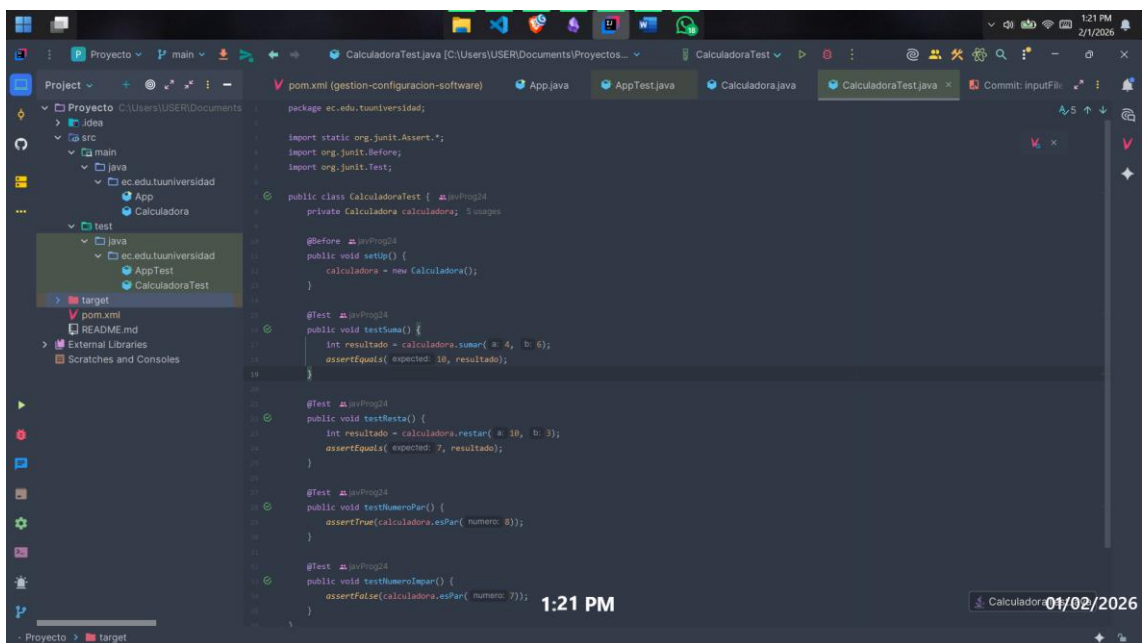
El desarrollo de este proyecto permitió comprender la importancia de la gestión de la configuración del software dentro del proceso de desarrollo. El uso de Maven facilitó la automatización de tareas repetitivas y promovió una estructura ordenada del proyecto.

Además, la implementación de pruebas unitarias resaltó la relevancia de aplicar buenas prácticas de aseguramiento de calidad desde las primeras etapas del desarrollo. En conclusión, Apache Maven se presenta como una herramienta clave para el desarrollo de aplicaciones Java modernas, contribuyendo a la eficiencia, calidad y mantenibilidad del software.

11. Anexos

EVIDENCIAS A INSERTAR:

- Capturas del proyecto en IntelliJ IDEA



- Capturas de la ejecución de comandos Maven

The screenshot shows the Maven build process for a project named 'gestion-configuracion-software'. The terminal output includes the following details:

- Project:** gestion-configuracion-software
- Build Phase:** validate
- Warnings:** A deprecated method in sun.misc.Unsafe has been called.
- Build Success:** The build completed successfully.
- Test Results:** 4 tests passed, 4 tests total, 20 ms.
- Test Details:**
 - testNumeroPar: 18 ms
 - testResta: 0 ms
 - testSuma: 0 ms
 - testNumeroImpar: 2 ms

- Capturas de pruebas unitarias

The screenshot shows the Run tab in an IDE, displaying the execution of unit tests for the 'CalculadoraTest' class. The results are as follows:

- Test Class:** CalculadoraTest
- Test Results:** 4 tests passed, 4 tests total, 20 ms.
- Test Details:**
 - testNumeroPar: 18 ms
 - testResta: 0 ms
 - testSuma: 0 ms
 - testNumeroImpar: 2 ms

- Enlace al repositorio GitHub: <https://github.com/Peter16Xo/gestiongrupob.git>
- Enlace al video de presentación en Drive: <https://drive.google.com/file/d/10evL4QavEAQVgjIF0JalS68lXy4nDZpV/view?usp=sharing>